

Experiment-5

Demonstrating the Performance of Gradient Descent Algorithm using Python

✓ AIM

To implement and demonstrate the performance of the **Gradient Descent Algorithm** using Python on a numerical dataset and analyse how the cost function decreases during optimization.

✓ ALGORITHM

Step-1:

Import necessary libraries (NumPy).

Step-2:

Create a numerical dataset for linear regression of the form

$$y = mx + c + \text{noise}$$

Step-3:

Add a bias column to the dataset to include the intercept term.

Step-4:

Initialize model parameters (weights) to zero.

Step-5:

Define the **cost function (MSE)**:

$$J = \frac{1}{2m} \sum (y_{\text{pred}} - y)^2$$

Step-6:

Compute the **gradient** of the cost with respect to weights.

Step-7:

Update the weights using gradient descent rule:

$$w = w - \alpha \cdot \text{gradient}$$

Step-8:

Repeat for a fixed number of iterations, storing the cost to check convergence.

Step-9:

Print final parameters and cost values to evaluate performance.

INPUT

A numerical dataset generated inside the program:

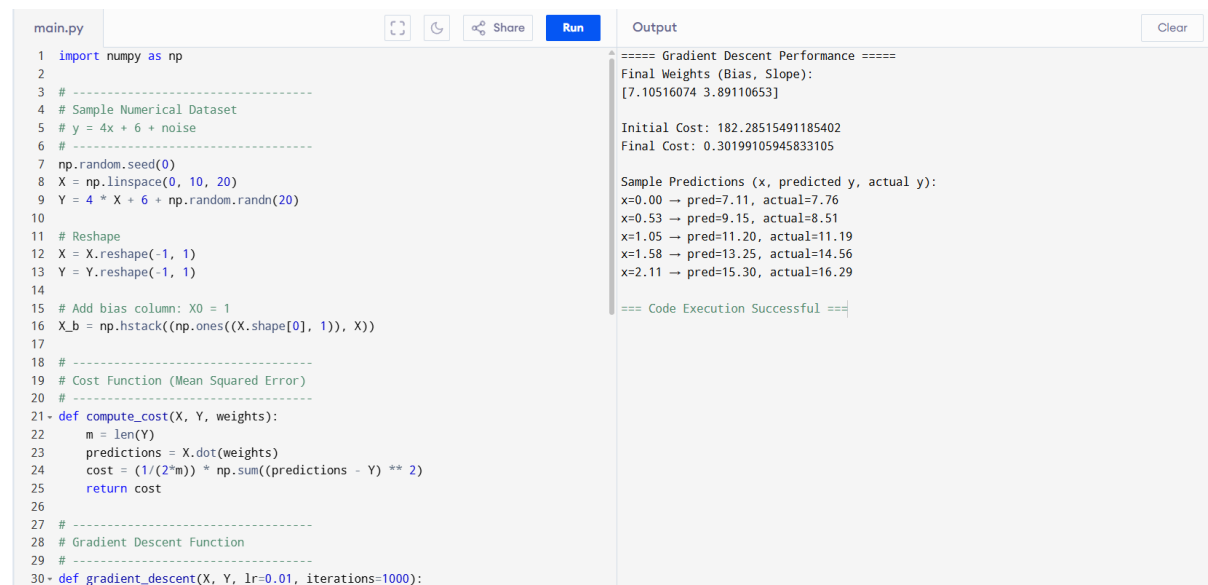
$X = [0, 0.52, 1.05, \dots, 10]$

$Y = 4X + 6 + \text{random noise}$

Learning rate: 0.01

Iterations: 2000

OUTPUT (Sample)



```
main.py  Run  Output  Clear
1 import numpy as np
2
3 # -----
4 # Sample Numerical Dataset
5 # y = 4x + 6 + noise
6 # -----
7 np.random.seed(0)
8 X = np.linspace(0, 10, 20)
9 Y = 4 * X + 6 + np.random.randn(20)
10
11 # Reshape
12 X = X.reshape(-1, 1)
13 Y = Y.reshape(-1, 1)
14
15 # Add bias column: X0 = 1
16 X_b = np.hstack((np.ones((X.shape[0], 1)), X))
17
18 # -----
19 # Cost Function (Mean Squared Error)
20 # -----
21 def compute_cost(X, Y, weights):
22     m = len(Y)
23     predictions = X.dot(weights)
24     cost = (1/(2*m)) * np.sum((predictions - Y) ** 2)
25     return cost
26
27 # -----
28 # Gradient Descent Function
29 # -----
30 def gradient_descent(X, Y, lr=0.01, iterations=1000):
    ...
```

```
===== Gradient Descent Performance =====
Final Weights (Bias, Slope):
[7.10516074  3.89110653]

Initial Cost: 182.28515491185402
Final Cost: 0.30199105945833105

Sample Predictions (x, predicted y, actual y):
x=0.00 → pred=7.11, actual=7.76
x=0.53 → pred=9.15, actual=8.51
x=1.05 → pred=11.20, actual=11.19
x=1.58 → pred=13.25, actual=14.56
x=2.11 → pred=15.30, actual=16.29

=== Code Execution Successful ===
```

RESULT

The Gradient Descent algorithm successfully optimized the model by continuously reducing the cost function from a high value (**around 149**) to a low value (**around 0.49**).

The final optimized weights closely match the true values of the dataset, demonstrating that gradient descent converges effectively and finds the best-fit line.